

Auditer et analyser l'activité de Claude Code via les logs

Le registre JSONL : source de vérité

Chaque action de Claude Code est enregistrée dans des fichiers JSONL (une entrée JSON par ligne) stockés dans `~/.claude/projects/<projet>/`. Ces fichiers sont la source de vérité sur ce que l'agent a réellement fait : quels fichiers il a lus, quels fichiers il a modifiés, quelles commandes il a exécutées.

```
# Localiser les fichiers de session du projet courant
ls
~/.claude/projects/- $( pwd | tr '/' '-' )
) -/ # → SESSION_ID.jsonl SESSION_ID2.jsonl ...
```

Chaque ligne de type `assistant` contient un tableau `content` dont les entrées `tool_use` documentent chaque appel d'outil avec ses paramètres d'entrée.

Ce que les outils exposent

Outil	Information disponible
Read	Chemin du fichier, plage de lignes
Edit / Write	Chemin, contenu modifié
Bash	Commande complète exécutée
WebFetch	URL appelée (données POST incluses)
Task	Prompt transmis au sous-agent
Glob / Grep	Patterns de recherche et scope

Requêtes jq essentielles

```
SESSION = ~/.claude/projects/-mon-projet-/SESSION.jsonl
# Fichiers lus dans la session
jq 'select(.type=="assistant") | .message.content[]? | select(.type=="tool_use" and .name=="Read") | .input.file_path' "$SESSION"
# Commandes bash exécutées
jq 'select(.type=="assistant") | .message.content[]? | select(.type=="tool_use" and .name=="Bash") | .input.command' "$SESSION"
# Comptage par outil (profil d'activité)
jq -r 'select(.type=="assistant") | .message.content[]? | select(.type=="tool_use") | .name' \
$SESSION | sort | uniq -c | sort -rn
```

Patterns sensibles à surveiller

Ces patterns valent la peine d'être détectés automatiquement dans les audits :

Pattern	Risque	Filtre jq
Read sur <code>.env</code> , <code>*.pem</code> , <code>id_rsa</code>	Accès aux credentials	<code>test("\\.(env pem key)\$")</code>
Bash avec <code>rm -rf</code> ou <code>--force-push</code>	Action destructive	<code>test("rm -rf")</code>
WebFetch vers URL externe	Exfiltration potentielle	sélectionner <code>.name=="WebFetch"</code>

Pattern	Risque	Filtre jq
Write hors du répertoire projet	Scope creep	Comparer le chemin au répertoire courant

Pour les équipes, synchroniser ces logs vers un stockage immuable (S3 avec versioning, par exemple) pour conserver une trace forensique inaltérable.

Outils externes de la communauté

Outil	Type	Usage principal
ccusage	CLI / TUI	Coûts depuis JSONL, référence communauté
claude-code-otel	OpenTelemetry	Export vers Prometheus + Grafana
ccboard	TUI + Web	Dashboard sessions, coûts, activité
Akto	SaaS	Guardrails sécurité au niveau API
MLflow Tracing	CLI + SDK	Token counts exacts, évaluation LLM

```
# Installation rapide
npm i -g ccusage
# Coûts et statistiques
cargo install ccboard
# Dashboard interactif
npm i -g claude-code-otel
# Export OpenTelemetry
```

Arbre de décision

Besoin rapide de coûts ? → ccusage (0 config)

Audit entreprise ? → claude-code-otel + Grafana

Déjà sur MLflow ? → MLflow tracing integration

Dashboard visuel ? → ccboard

Estimation des coûts dans le JSONL

Les logs JSONL ne contiennent pas les prix Anthropic directement. Les outils comme ccusage appliquent une heuristique d'environ 4 caractères par token, puis multiplient par les tarifs du modèle. Cette estimation tend à légèrement surestimer ; les chiffres sont indicatifs, pas facturés.